

SecurePress — Project Plan

Vision

SecurePress aims to become a modern security infrastructure layer for WordPress inspired by Laravel's architecture and developer experience.

Instead of competing as another traditional firewall or malware scanner, SecurePress will focus on:

- Middleware-based security
- Developer-first APIs
- Authentication hardening
- Rate limiting
- Signed URLs
- Secure request handling
- Audit logging
- WooCommerce security
- Modern architecture for WordPress

Core positioning:

"Laravel-style security architecture for WordPress."

1. Product Goals

Primary Goals

- Build lightweight, modular security infrastructure for WordPress.
- Improve developer experience for security handling.
- Reduce dependency on bloated security plugins.
- Provide modern APIs similar to Laravel.
- Offer WooCommerce-focused protection.
- Create a scalable SaaS opportunity.

Non-Goals (Initially)

Avoid building these in the first versions:

- Enterprise firewall
 - Heavy malware scanning engine
 - Antivirus competitor
 - CDN-level protection
 - Cloudflare replacement
-

2. Product Positioning

Target Users

Primary Audience

- WordPress developers
- WooCommerce developers
- Agencies
- Freelancers
- Technical site owners

Secondary Audience

- Store owners
 - Membership websites
 - LMS websites
 - SaaS businesses using WordPress
-

3. High-Level Product Architecture

Technology Stack

Backend

- PHP 8.2+
- Composer
- PSR-4 autoloading
- Dependency Injection Container
- REST API
- WordPress Hooks
- Monolog
- Symfony Components

Frontend

- React
- WordPress Gutenberg Components
- Tailwind CSS (optional)

Testing

- PHPUnit
- PestPHP

Suggested Libraries

- monolog/monolog
- symfony/http-foundation

- symfony/rate-limiter
 - vlucas/phpdotenv
 - paragonie/random_compat
-

4. Proposed Folder Structure

```
securepress/  
├─ bootstrap/  
├─ config/  
├─ resources/  
├─ routes/  
├─ src/  
│   ├─ Core/  
│   ├─ Middleware/  
│   ├─ Security/  
│   ├─ Auth/  
│   ├─ Logging/  
│   ├─ Validation/  
│   ├─ Http/  
│   ├─ Admin/  
│   └─ WooCommerce/  
├─ storage/  
│   ├─ logs/  
│   └─ cache/  
├─ tests/  
├─ vendor/  
├─ securepress.php  
└─ composer.json
```

5. Core System Design

5.1 Service Container

Laravel-inspired dependency injection container.

Responsibilities:

- Register services
- Resolve dependencies
- Manage modules
- Enable extensibility

Example:

```
$app->bind(LoggerInterface::class, MonologLogger::class);
```

5.2 Middleware Pipeline

The middleware system becomes the foundation of SecurePress.

Example:

```
Security::middleware([
    RateLimit::class,
    CsrfProtection::class,
    BotProtection::class,
]);
```

Initial Middleware Ideas

- RateLimitMiddleware
 - CsrfMiddleware
 - AdminProtectionMiddleware
 - BotDetectionMiddleware
 - MaintenanceMiddleware
 - SignedUrlMiddleware
 - GeoRestrictionMiddleware (future)
-

5.3 Event System

Internal event-driven architecture.

Events:

- UserLoginEvent
- FailedLoginEvent
- PluginUpdatedEvent
- FileModifiedEvent
- AdminActionEvent

Benefits:

- Modular design
 - Easier extensions
 - Future SaaS integration
-

6. MVP Scope (Version 1.0)

Core Features

1. Login Protection

Features:

- Brute-force protection
 - Login rate limiting
 - Failed login detection
 - Temporary IP blocking
 - Email alerts
-

2. Rate Limiting Engine

Protect:

- wp-login.php
- XML-RPC
- REST API
- WooCommerce checkout
- Contact forms

Example API:

```
RateLimiter::for('login', 5, 'minute');
```

3. Security Headers

Headers:

- Content-Security-Policy
- Strict-Transport-Security
- X-Frame-Options
- Referrer-Policy
- Permissions-Policy

Admin UI:

- Toggle-based configuration
 - Preset profiles
-

4. Signed URLs

Example:

```
Security::signedUrl('/download/123', expires: 3600);
```

Use Cases:

- Secure downloads
 - Temporary access links
 - Password resets
 - Invite links
-

5. CSRF Protection Layer

Improve WordPress nonce handling.

Features:

- Expiring tokens
 - Middleware validation
 - Secure token generation
 - Form protection APIs
-

6. Audit Logging

Track:

- Login attempts
 - Plugin changes
 - Role updates
 - Admin actions
 - File editor usage
 - WooCommerce events
-

7. Developer SDK

Developer-focused APIs.

Example:

```
Security::protectRoute('/admin/export');
```

```
Security::rateLimit('checkout', 10, 'minute');
```

7. Version 1.1 Roadmap

Additional Features

Two-Factor Authentication

Support:

- TOTP apps
- Email OTP
- Backup codes

Device & Session Management

Features:

- Active session tracking
- Remote logout
- Device alerts

WooCommerce Security Pack

Features:

- Fake checkout prevention
- Bot checkout protection
- Coupon abuse protection
- Registration abuse detection
- API abuse throttling

Validation Layer

Laravel-style validation.

Example:

```
Validator::make($_POST, [  
    'email' => 'required|email'  
]);
```

8. Version 2.0 Roadmap

Advanced Security Features

File Integrity Monitoring

Track:

- Core changes
- Plugin modifications
- Theme modifications
- Unexpected PHP files

Malware Detection

Basic scanning:

- eval() patterns
- base64 obfuscation
- shell signatures
- suspicious injections

Threat Intelligence

Cloud-powered:

- Shared IP reputation
- Global attack patterns
- Centralized analytics

9. SaaS Roadmap

Central Dashboard

Allow agencies to manage multiple websites.

Features:

- Central monitoring
- Attack analytics
- Audit logs
- Alerts
- Remote controls

- Security reports
-

Recurring Revenue Opportunities

Agency Plans

- Multi-site management
- Advanced analytics
- Team access

Premium Modules

- WooCommerce Security
 - Threat Intelligence
 - Advanced Logging
 - Geo Blocking
 - AI-assisted anomaly detection
-

10. Development Timeline

Phase 1 — Architecture (Weeks 1-2)

Tasks

- Setup Composer
- Setup PSR-4 autoloading
- Build plugin bootstrap
- Create service container
- Setup configuration system
- Setup logging
- Create admin UI skeleton

Deliverable:

- Functional plugin foundation
-

Phase 2 — Security Core (Weeks 3-5)

Tasks

- Build middleware pipeline
- Implement rate limiter
- Implement login protection
- Add security headers

- Create signed URLs
- Create CSRF middleware

Deliverable:

- MVP security engine
-

Phase 3 — Developer APIs (Weeks 6–7)

Tasks

- Create SDK
- Add helper functions
- Create validation layer
- Add event system
- Create extension architecture

Deliverable:

- Developer-ready APIs
-

Phase 4 — Logging & Analytics (Week 8)

Tasks

- Build audit logging
- Add dashboards
- Create log viewer
- Export functionality

Deliverable:

- Monitoring system
-

Phase 5 — Beta Launch (Weeks 9–10)

Tasks

- Documentation
- Performance optimization
- Security testing
- Beta website
- Landing page
- GitHub repository

Deliverable:

- Public beta release
-

11. Suggested Pricing Strategy

Free Version

Include:

- Middleware engine
- Basic rate limiting
- Security headers
- Audit logging
- Signed URLs

Goal:

- Adoption
 - Community growth
 - Developer trust
-

Pro Version

Include:

- 2FA
- Advanced analytics
- WooCommerce protection
- Device management
- Cloud sync
- Advanced rules

Suggested Pricing:

- \$49/year single site
 - \$149/year agency
 - \$399/year unlimited
-

12. Marketing Roadmap

Content Strategy

Topics:

- Modern WordPress security
 - Laravel concepts for WordPress
 - Middleware architecture
 - Secure WooCommerce development
 - Developer-first plugins
-

SEO Opportunities

Potential keywords:

- WordPress middleware
 - WordPress rate limiting
 - WordPress signed URLs
 - WooCommerce security plugin
 - Laravel for WordPress
-

Community Strategy

- Open-source core
 - GitHub visibility
 - Developer documentation
 - YouTube tutorials
 - Reddit engagement
 - Facebook WordPress groups
-

13. Technical Risks

Risks

- WordPress compatibility issues
- Performance overhead
- Plugin conflicts
- Shared hosting limitations
- Complex middleware interactions

Mitigation

- Extensive testing

- Modular architecture
 - Fallback compatibility layers
 - Lightweight request handling
-

14. Success Metrics

Early Metrics

- GitHub stars
- WordPress installs
- Agency adoption
- WooCommerce adoption
- Active developers using APIs

Business Metrics

- Monthly recurring revenue
 - Churn rate
 - Average revenue per user
 - Conversion rate from free to pro
-

15. Long-Term Vision

SecurePress evolves into:

"The modern security and infrastructure framework for WordPress applications."

Long-term possibilities:

- Headless WordPress security
 - API gateway layer
 - Cloud-managed security
 - Enterprise compliance modules
 - Managed hosting partnerships
 - Developer ecosystem and marketplace
-

16. Recommended Immediate Next Steps

Week 1 Action Plan

Step 1

Create repository:

- GitHub organization
- Plugin repository
- Composer setup

Step 2

Build architecture foundation:

- PSR-4
- Container
- Bootstrap process
- Config system

Step 3

Build middleware prototype:

- Request interception
- Pipeline handling
- Rate limiting proof of concept

Step 4

Build admin dashboard skeleton:

- Plugin settings
- Logs page
- Security overview

Step 5

Create branding:

- Logo
- Landing page
- Documentation structure

Final Recommendation

The biggest opportunity is NOT building another WordPress firewall.

The opportunity is:

- modern architecture
- developer experience
- WooCommerce security
- middleware-based protection
- Laravel-inspired APIs

If executed correctly, SecurePress can become:

"The Laravel-style security framework for WordPress developers."